# Build a Student Group Collaboration App

With Tracking, Contributions & Version Control

■ Analytics ■ Time Track ■ Git History ■ Team Work

| VERSION | STACK | YEAR |
| --- | --- | --- |
| **1.0** | **React + Node + PostgreSQL** | **2025** |

# Table of Contents

# 1 Project Overview & Goals

## What You Are Building

A full-stack web application that enables students to collaborate on group projects (documents and presentations) in real time, while automatically tracking each student's contributions, time spent, and maintaining a complete version history of every change made.

| | |
|---|---|
| ■ **Collaborative Editing** | Multiple students edit the same document/presentation simultaneously, like Google Docs. |
| ■ **Contribution Dashboard** | Visual charts showing how much each student contributed — words written, edits made, sections owned. |
| ■ **Time Tracking** | Automatic timers log how long each student is actively working in the app per session. |
| ■ **Version Control** | Every save creates a snapshot. Roll back to any previous version and see a full diff of changes. |
| ■■■ **Teacher View** | Teachers get a separate dashboard to monitor all groups — who is contributing and who is not. |
| ■ **Notifications** | Alerts when teammates make changes or when a deadline is approaching. |

■ *All features work together: when a student edits a document, their contribution score increases, their active time is logged, and a new version snapshot is created automatically.*

# 2 Tech Stack & Architecture

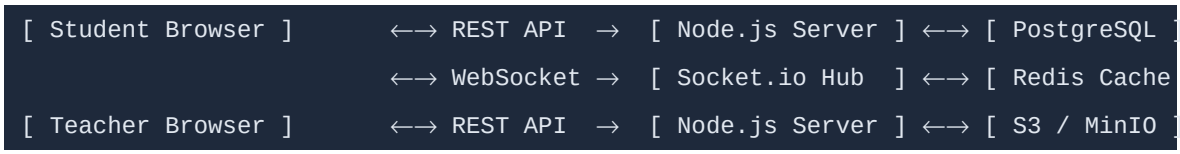
## Recommended Technology Stack

The following stack is chosen for reliability, developer-friendliness, and strong ecosystem support for real-time features and data analytics:

| Layer | Technology | Purpose |
|---|---|---|
| Frontend | React 18 + Vite | UI, routing, state management |
| Editor | TipTap (ProseMirror) | Rich text collaborative editor with diff support |
| Slides | Reveal.js | Presentation editor component |
| State | Zustand + React Query | Global state & server-state caching |
| Backend | Node.js + Express | REST API server |
| Real-time | Socket.io | WebSocket for live collaboration |
| Database | PostgreSQL | Primary data store (users, projects, versions) |
| Cache | Redis | Active sessions, presence data, rate limiting |
| Auth | JWT + bcrypt | Secure authentication and token refresh |
| File Store | AWS S3 / MinIO | Store presentation assets, images, exports |
| Hosting | Docker + Nginx | Containerised deployment |

## System Architecture Diagram (Conceptual)

The diagram below shows how the components communicate with each other:



# 3 Database Schema Design

## Core Database Tables

| Table Name | Key Columns |
|---|---|
| `users` | id, name, email, password_hash, role (student/teacher), created_at |
| `groups` | id, name, project_title, teacher_id, deadline, created_at |
| `group_members` | id, group_id, user_id, joined_at |
| `documents` | id, group_id, title, type (doc/presentation), current_version_id, created_at |
| `document_versions` | id, document_id, content_json, version_number, created_by, created_at, change_summary |
| `contributions` | id, document_id, user_id, words_added, words_deleted, edits_count, recorded_at |
| `time_sessions` | id, user_id, document_id, start_time, end_time, duration_seconds |
| `activity_log` | id, user_id, document_id, action_type, metadata_json, created_at |

### SQL: Create Contributions Table

```
CREATE TABLE contributions ( id SERIAL PRIMARY KEY, document_id INTEGER REFERENCES documents(id), user_id INTEGER REFERENCES users(id), words_added INTEGER DEFAULT 0, words_deleted INTEGER DEFAULT 0, edits_count INTEGER DEFAULT 0, recorded_at TIMESTAMP DEFAULT NOW() ); CREATE INDEX idx_contrib_doc ON contributions(document_id); CREATE INDEX idx_contrib_user ON contributions(user_id);
```

# 4 Backend API Development (Node.js)

## REST API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| POST | /api/auth/register | Register new student or teacher |
| POST | /api/auth/login | Login and receive JWT token |
| GET | /api/groups/:id/members | List all group members |
| POST | /api/documents | Create new document or presentation |
| GET | /api/documents/:id | Get document with latest version |
| POST | /api/documents/:id/versions | Save new version (snapshot) |
| GET | /api/documents/:id/versions | List all versions (history) |
| GET | /api/documents/:id/contributions | Contribution stats per student |
| GET | /api/documents/:id/time-sessions | Time spent per student |
| POST | /api/documents/:id/time-sessions | Start/stop a time tracking session |

### Save Version Endpoint (Node.js + Express)

```
// POST /api/documents/:id/versions router.post('/:id/versions', authenticate, async (req, res) => { const { content, changeSummary } = req.body; const userId = req.user.id; const docId = req.params.id; // 1. Get the latest version number const last = await db.query( 'SELECT version_number FROM document_versions WHERE document_id=$1 ORDER BY version_number DESC LIMIT 1', [docId]); const nextVer = (last.rows[0]?.version_number ?? 0) + 1; // 2. Insert new version snapshot await db.query( `INSERT INTO document_versions (document_id, content_json, version_number, created_by, change_summary) VALUES ($1,$2,$3,$4,$5)`, [docId, JSON.stringify(content), nextVer, userId, changeSummary]); // 3. Calculate word diff and record contribution const prev = await getPreviousContent(docId, nextVer - 1); const diff = computeDiff(prev, content); await recordContribution(docId, userId, diff); res.json({ version: nextVer, message: 'Version saved' }); });
```

# 5 Frontend Development (React)

## Project Folder Structure

```
src/ ■■■ components/ ■ ■■■ Editor/ ■ ■ ■■■ DocumentEditor.jsx ← TipTap rich text editor ■ ■ ■■■ PresentationEditor.jsx ■ ■ ■■■ VersionHistory.jsx ← diff viewer + restore ■ ■■■ Dashboard/ ■ ■ ■■■ ContributionChart.jsx ← bar chart per student ■ ■ ■■■ TimeTracker.jsx ← live timer display ■ ■ ■■■ ActivityFeed.jsx ← recent edits feed ■ ■■■ Teacher/ ■ ■ ■■■ GroupOverview.jsx ■ ■ ■■■ StudentReport.jsx ■ ■■■ shared/ ■ ■■■ Avatar.jsx ■ ■■■ PresenceIndicator.jsx ← shows who's online ■■■ hooks/ ■ ■■■ useSocket.js ← WebSocket connection hook ■ ■■■ useTimer.js ← active session timer ■ ■■■ useVersions.js ← fetch / restore versions ■■■ pages/ ■ ■■■ Dashboard.jsx ■ ■■■ Document.jsx ■ ■■■ TeacherPanel.jsx ■■■ store/ ■■■ useAppStore.js ← Zustand global state
```

### Contribution Chart Component (React + Recharts)

```
import { BarChart, Bar, XAxis, YAxis, Tooltip, Legend } from 'recharts'; import { useQuery } from '@tanstack/react-query'; export function ContributionChart({ documentId }) { const { data } = useQuery({ queryKey: ['contributions', documentId], queryFn: () => fetch(`/api/documents/${documentId}/contributions`) .then(r => r.json()) }); return ( <BarChart width={500} height={300} data={data}> <XAxis dataKey="studentName" /> <YAxis /> <Tooltip /> <Legend /> <Bar dataKey="wordsAdded" fill="#4F46E5" name="Words Added" /> <Bar dataKey="editsCount" fill="#06B6D4" name="Edits Made" /> <Bar dataKey="timeMinutes" fill="#10B981" name="Time (min)" /> </BarChart> ); }
```

# 6 Version Control System

## How Version Control Works in This App

Unlike Git (which tracks code files line by line), this app tracks rich document content as JSON snapshots. Every time a student saves (or auto-save fires), a new version is stored in the database with the full content and a diff summary.

| | |
|---|---|
| **Step 1 — Auto-Save Trigger** | Every 30 seconds OR after 50+ characters changed, the editor auto-saves. |
| **Step 2 — Diff Calculation** | The server compares the new content JSON vs. the previous version to find added/deleted text. |
| **Step 3 — Snapshot Storage** | A full content snapshot is stored in document_versions with version number, author, and timestamp. |
| **Step 4 — History UI** | Students/teachers can view a timeline of all versions. Clicking any version shows a colour-coded diff. |
| **Step 5 — Restore** | Any version can be restored with one click. A new version is created (restore is non-destructive). |

### Get Version History API Handler

```
// GET /api/documents/:id/versions router.get('/:id/versions', authenticate, async (req, res) => { const versions = await db.query(` SELECT dv.id, dv.version_number, dv.change_summary, dv.created_at, u.name AS author_name, u.avatar_url FROM document_versions dv JOIN users u ON u.id = dv.created_by WHERE dv.document_id = $1 ORDER BY dv.version_number DESC `, [req.params.id]); res.json(versions.rows); }); // GET /api/documents/:id/versions/:vId/diff — returns colour diff router.get('/:id/versions/:vId/diff', authenticate, async (req,res) => { const curr = await getVersion(req.params.vId); const prev = await getVersion(curr.version_number - 1, req.params.id); const diff = diffWords(prev.content_json, curr.content_json); res.json({ diff }); });
```

# 7 Contribution Tracking Engine

## What Gets Tracked Per Student

| Metric | How Measured | Stored In |
|---|---|---|
| Words Added | Word count diff between versions | contributions.words_added |
| Words Deleted | Negative diff word count | contributions.words_deleted |
| Edit Events | Every keypress batch = 1 edit | contributions.edits_count |
| Sections Owned | Sections where student made >50% edits | Computed on read |
| Last Active | Timestamp of most recent action | activity_log |
| Contribution % | Student edits / total group edits | Computed on read |

### Contribution Score Calculation Logic

```
// Compute contribution percentage for each group member async function
getContributionStats(documentId) { const rows = await db.query(` SELECT u.id, u.name,
SUM(c.words_added) AS words_added, SUM(c.words_deleted) AS words_deleted, SUM(c.edits_count)
AS edits_count FROM contributions c JOIN users u ON u.id = c.user_id WHERE c.document_id = $1
GROUP BY u.id, u.name `, [documentId]); const total = rows.rows.reduce((s, r) => s +
r.edits_count, 0); return rows.rows.map(r => ({ ...r, contribution_pct: total > 0 ?
((r.edits_count / total) * 100).toFixed(1) : 0, })); }
```

# 8 Time Tracking Feature

## How Time is Tracked Accurately

Time tracking is split into two layers: active time (student is typing or clicking) and session time (tab is open). This ensures fair measurement of real effort.

- When a student opens a document, a time_session record is created with start_time.
- A heartbeat ping is sent every 60 seconds from the frontend to keep the session alive.
- If no heartbeat arrives for 3 minutes, the session is auto-closed (idle detection).
- When the student closes the tab or navigates away, end_time is recorded via the beforeunload event.
- The dashboard shows total time, active time, and a per-day breakdown chart.

### Frontend Timer Hook (React)

```
// hooks/useTimer.js import { useEffect, useRef } from 'react'; import { useMutation } from '@tanstack/react-query'; export function useDocumentTimer(documentId) { const sessionId = useRef(null); const startSession = useMutation({ mutationFn: () => fetch(`/api/documents/${documentId}/time-sessions`, { method: 'POST', body: JSON.stringify({ action: 'start' }) }) .then(r => r.json()), onSuccess: (data) => { sessionId.current = data.sessionId; } }); const endSession = useMutation({ mutationFn: () => fetch(`/api/documents/${documentId}/time-sessions`, { method: 'POST', body: JSON.stringify({ action:'end', sessionId: sessionId.current }) }) }); useEffect(() => { startSession.mutate(); // Heartbeat every 60s const hb = setInterval(() => { fetch(`/api/sessions/${sessionId.current}/heartbeat`, { method:'POST' }); }, 60000); window.addEventListener('beforeunload', () => endSession.mutate()); return () => { clearInterval(hb); endSession.mutate(); }; }, [documentId]); }
```

# 9 Real-Time Collaboration (WebSocket)

## Socket.io Events Architecture

| Event Name | Direction | Description |
|---|---|---|
| join-document | Client → Server | Student joins a document room |
| leave-document | Client → Server | Student leaves (tab closed) |
| doc-change | Client → Server | Student made an edit (sends delta) |
| doc-change | Server → Clients | Broadcast edit to all other members |
| user-presence | Server → Clients | Who is currently online in this doc |
| version-saved | Server → Clients | A new version was auto-saved |
| cursor-move | Client → Server | Student cursor position changed |
| cursor-positions | Server → Clients | All cursors for display |

### Server-Side Socket.io Handler

```
// server/sockets/documentSocket.js io.on('connection', (socket) => {
socket.on('join-document', ({ documentId, userId }) => { socket.join(`doc-${documentId}`);
// Notify others of new presence socket.to(`doc-${documentId}`).emit('user-presence', {
type: 'joined', userId }); }); socket.on('doc-change', async ({ documentId, delta, userId })
=> { // Broadcast to all OTHER members in the room
socket.to(`doc-${documentId}`).emit('doc-change', { delta, userId }); // Record edit event
for contribution tracking await recordEditEvent(documentId, userId, delta); });
socket.on('disconnect', () => { // Remove from all presence lists
broadcastUserLeft(socket.id); endActiveTimerSession(socket.id); }); });
```

# 10 Deployment & DevOps

## Docker Compose Setup

```
# docker-compose.yml version: '3.9' services: db: image: postgres:16 environment: POSTGRES_DB: collab_db POSTGRES_USER: admin POSTGRES_PASSWORD: secret volumes: - pgdata:/var/lib/postgresql/data redis: image: redis:7-alpine backend: build: ./backend ports: ["4000:4000"] environment: DATABASE_URL: postgres://admin:secret@db/collab_db REDIS_URL: redis://redis:6379 JWT_SECRET: your-secret-key depends_on: [db, redis] frontend: build: ./frontend ports: ["3000:80"] depends_on: [backend] volumes: pgdata:
```

## Recommended Deployment Steps

| | |
|---|---|
| **1. Local Dev** | Run docker-compose up — all services start together. |
| **2. Database Init** | Run migration scripts: node scripts/migrate.js |
| **3. Seed Data** | Create test teacher account and student group: node scripts/seed.js |
| **4. Frontend Build** | npm run build inside /frontend — outputs to /frontend/dist |
| **5. Production** | Push to VPS / cloud (DigitalOcean, AWS EC2). Use Nginx to proxy port 80 → 3000 & 4000. |
| **6. SSL** | Use Certbot (Let's Encrypt) for free HTTPS certificate. |
| **7. Monitoring** | Add UptimeRobot for uptime alerts + basic logging with Morgan. |

■ *You now have a complete blueprint to build a Student Group Collaboration App with real-time editing, contribution tracking, time tracking, and version control. Start with Section 3 (Database Schema) and Section 4 (Backend API) first — then build the React frontend around the API. Good luck!*